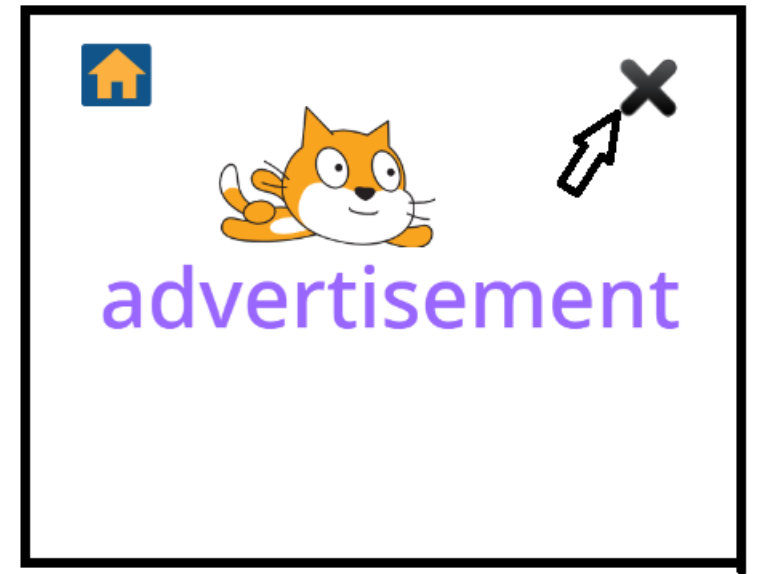


Improving User Interface Click System With AI

2024044620 김종진

- There are many ads at websites.
- When you miss-click ad,
- your browser jumps into ad-site.
- So I want to use ai to predict user's intention.

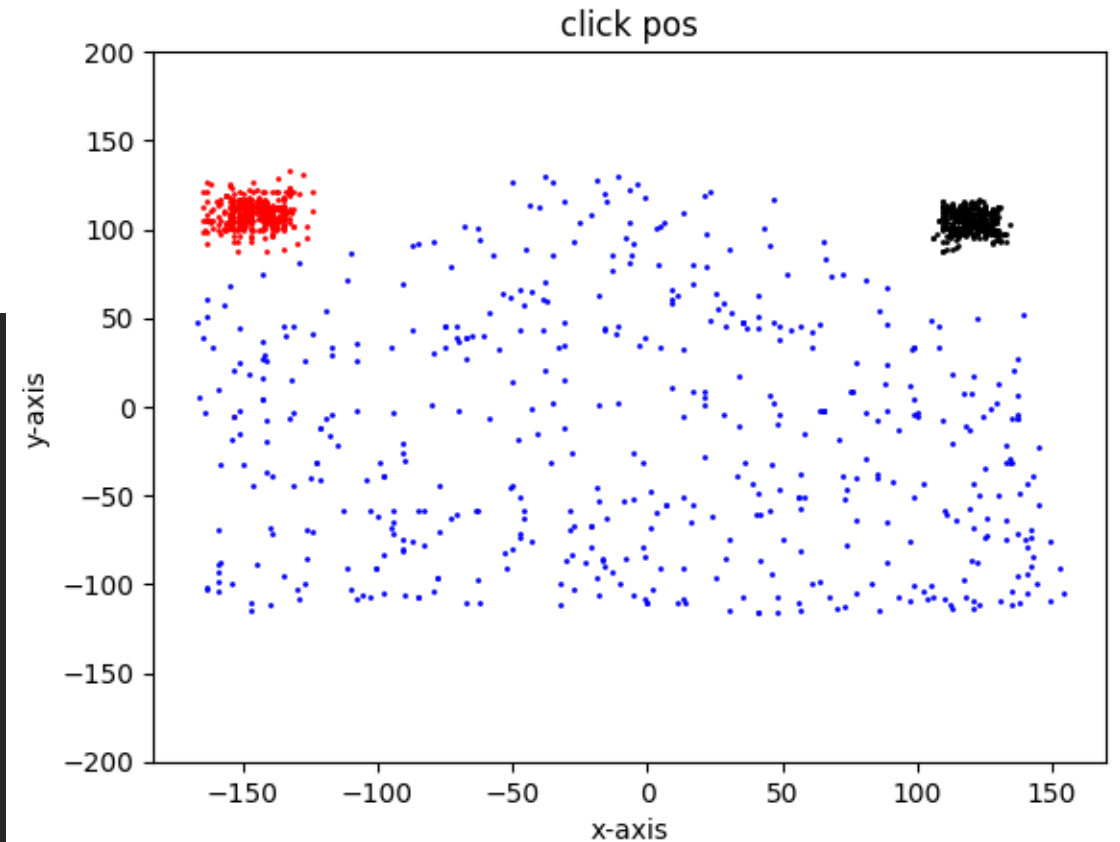
- Dismiss(X) button -> 'a'
 - Home button -> 'b'
 - Advertisement -> 'c'
-
- Click -> ai -> 'a' or 'b' or 'c'



Jump to random ad site

- First, I need to collect data.
- Red -> 'a'
- Black -> 'b'
- Blue -> 'c'
- Each element has 500 data.
- Click -> xpos:ypox:element
- I added it manually.

```
118:100:a  
121:104:a  
121:105:a  
117:104:a  
114:108:a  
112:110:a  
115:107:a  
116:107:a  
-156:119:b  
-156:119:b  
-156:119:b  
-153:120:b  
-150:120:b  
-146:121:b  
-142:121:b  
-137:121:b  
-134:121:b
```



- I used random forest method for ai
- to distinguish user's intention.
- You can find out more information
- about random forest at Wikipedia.
- https://en.wikipedia.org/wiki/Random_forest

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import LabelEncoder

def load_data(file_path):
    X = []
    y = []
    with open(file_path, "r") as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            x, y_val, label = line.split(":")
            X.append([float(x), float(y_val)])
            y.append(label)
    return np.array(X), np.array(y)

def train_model(X, y, model_type="rf"):
    le = LabelEncoder()
    y_encoded = le.fit_transform(y)
    X_train, X_test, y_train, y_test = train_test_split(
        X, y_encoded, test_size=0.2, random_state=42
    )
    model = RandomForestClassifier(n_estimators=100, random_state=42)
    model.fit(X_train, y_train)
    accuracy = model.score(X_test, y_test)
    print("accuracy: " + str(accuracy))
    return model, le

def predict(model, le, x, y):
    pred = model.predict([[x, y]])
    label = le.inverse_transform(pred)
    return label[0]

X, y = load_data("x,y,e.txt")
model, le = train_model(X, y, model_type="rf")
```

- So I made the AI model,
- AI can distinguish user's
- intention of click.
- And also this code is extremely fast,
- You don't need to wait.

```
[8] ✓ 2초
import time
xpos=int(input())
ypos=int(input())
start_time = time.perf_counter()
print(predict(model, le, xpos, ypos))
end_time = time.perf_counter()
execution_time = end_time - start_time
print(f"실행 시간: {execution_time:.6f}초")

... 30
50
c
실행 시간: 0.008224초
```

```
[4] ✓ 3초
xpos=int(input())
ypos=int(input())
print(predict(model, le, xpos, ypos))

... 110
110
a

[7] ✓ 3초
xpos=int(input())
ypos=int(input())
print(predict(model, le, xpos, ypos))

... -120
100
b

[5] ✓ 7초
xpos=int(input())
ypos=int(input())
print(predict(model, le, xpos, ypos))

... 10
-30
c
```